

Campus safety companion matching system ▾

I want you to create a system for a Student Helper Safety App. The app is for university students to feel safer walking around campus, especially at later hours. I will give you requirements for the system and how it should behave, then you will implement the functionality. Requirements:

- The system shall allow students to send requests to helpers so that they can get accompanied around campus.
- The system shall allow helpers to receive the requests from students and accept them.
- The system shall prioritize requests from students to helpers which are within close proximity.
- The system shall ensure that helpers are not receiving conflicting requests.
- The system shall allow students to apply for becoming a helper.
- The system shall provide a chat room for matched students and helpers.
- The system shall provide only the necessary information about the student to the helper without exposing sensitive details about the student.

Implement the functionality, no UI, no unit tests.

Show less

Check frontend design skill >

The user asked for "no UI", so I'll focus purely on the backend system implementation. Let me build a well-structured Node.js backend with all the required functionality.

Ran 4 commands >

Let me build all the source files while npm installs in the background.

Created 13 files >

Now let's create all the route handlers:

Created 2 files >

Let me create the remaining files and then bundle everything up.

Created 6 files >

Now let's write comprehensive API documentation:

Created 2 files >

Now let's verify the complete structure and do a syntax check:

Ran 4 commands >

All 22 files pass syntax checks. Let's copy to outputs:

Ran a command, read a file >

All 22 source files pass syntax validation. Here's a summary of what was built:

Project Structure

```
src/
├─ config/env.js           # PORT, JWT_SECRET, PROXIMITY_RADIUS_KM
├─ models/
│   └─ store.js           # In-memory Maps (users, requests,
matches, chatRooms, applications)
│   └─ user.js            # Roles, helperStatus,
toSafeStudentProfile()
│   └─ request.js         # EscortRequest + status enum
│   └─ match.js           # Match, ChatRoom, Message factories
│   └─ helperApplication.js # Application + status enum
├─ services/
│   └─ authService.js     # Register, login, JWT
│   └─ requestService.js  # Full request lifecycle + proximity
sort
│   └─ helperApplicationService.js # Apply + admin review
│   └─ chatService.js     # Room auth + message persistence
│   └─ locationService.js  # GPS updates
├─ routes/                # Express routers (one per domain)
├─ middleware/auth.js     # JWT verify + requireRole()
├─ utils/geo.js           # Haversine formula +
sortHelpersByProximity()
├─ socketHandler.js       # Socket.IO real-time chat
├─ app.js                 # Express wiring
└─ server.js              # HTTP + Socket.IO bootstrap
```

How Each Requirement Is Met

Requirement	Mechanism
Students send requests	<code>POST /requests</code> — one active request per student
Helpers receive & accept requests	<code>GET /requests/pending</code> + <code>POST /requests/:id/accept</code>
Proximity prioritisation	Haversine sort in <code>requestService</code> — closest student first
No conflicting assignments	<code>helperStatus: BUSY</code> guard blocks a second accept until complete/cancel
Apply to become a helper	<code>POST /helper-applications</code> → admin approves via <code>PATCH .../review</code> → role promoted
Chat room per match	Auto-created on accept; REST polling + Socket.IO real-time
Privacy — minimal student info	<code>toSafeStudentProfile()</code> exposes only id, displayName, photo, location, safetyNote — no email, phone, or studentId



Readme

Document · MD

Download



Api

Document · MD

Download

 Download all

make it in python

Created 24 files, ran 8 commands >

Now verify everything imports and is syntactically correct:

Ran 3 commands >

All modules import cleanly. Now do a quick end-to-end smoke test:

Ran 3 commands >

All 12 checks pass. Copy to outputs:

Ran a command, created a file, read a file >

All 12 smoke-test assertions passed. Here's what was built:

Structure

```

app/
├─ config.py                                PORT, JWT_SECRET, PROXIMITY_RADIUS_KM
├─ models/
│   └─ store.py                            In-memory dicts (replaces a real DB)
│   └─ schemas.py                          Pydantic models, enums, all
request/response bodies
├─ user.py                                create_user(), to_safe_student_profile()
├─ request.py / match.py / helper_application.py
├─ services/
│   └─ auth_service.py                     bcrypt + JWT (HS256 via python-jose)
│   └─ request_service.py                  Full lifecycle + Haversine proximity sort
│   └─ helper_application_service.py        Apply / review / promote role
│   └─ chat_service.py                     Auth-guarded room messages
│   └─ location_service.py
├─ routes/                                FastAPI APIRouters (one per domain)
├─ middleware/auth.py                      get_current_user() + require_role() deps
├─ utils/geo.py                            haversine_distance(),
sort_helpers_by_proximity()
├─ socket_handler.py                       python-socketio async server
└─ main.py                                FastAPI + Socket.IO mounted as single ASGI
app

```

Key differences from the Node version: FastAPI's dependency injection replaces Express middleware, Pydantic handles all validation declaratively, and the Socket.IO + FastAPI apps are composed as a single ASGI app so one `uvicorn` process serves both. Interactive API docs are available automatically at `/docs`.



Readme
Document · MD

Download



Main
PY

Download



Requirements

Download

TXT

⬇ Download all

